

Today's Agenda :-

Starting 7:05

1. 2D Matrices Introduction
2. Iterating 2D Matrices
 - a) Row Wise
 - b) Column Wise
3. Printing Diagonals
4. Row To Column Zero.

Definition

Array :- Collection of same data types.

ex:- `int arr[5] = {1, 2, 3, 4, 5}`

Matrix:-

A 2D array that stores elements in a grid or table format, organised into rows & columns; acting as an array of arrays.

Declaration

Java :-

`int [][] mat = new int [N][M];`

Annotations:

- `int`: datatype
- `mat`: name of the var. variable
- `N`: No. of rows
- `M`: No. of columns

C++:-

`int mat [N][M];`

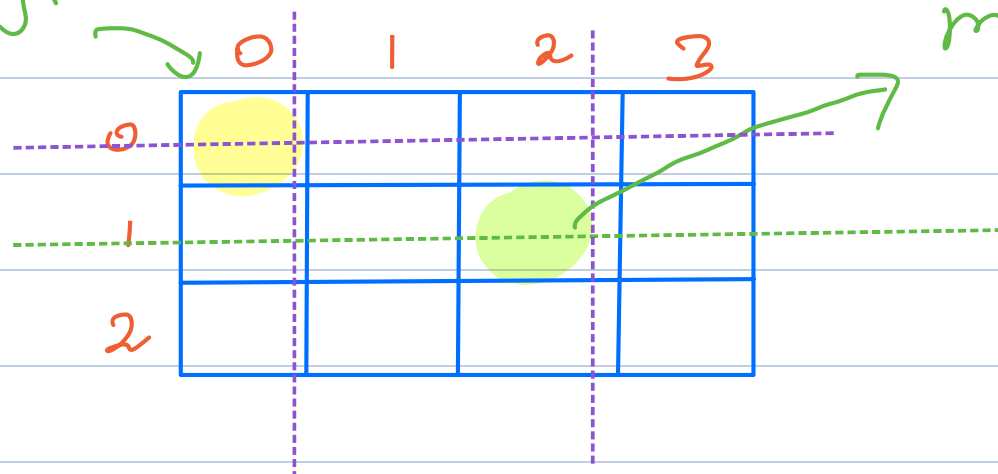
Python:-

`mat = [[0] * M for _ in range(N)]`

Ex:-

int mat[3][4]

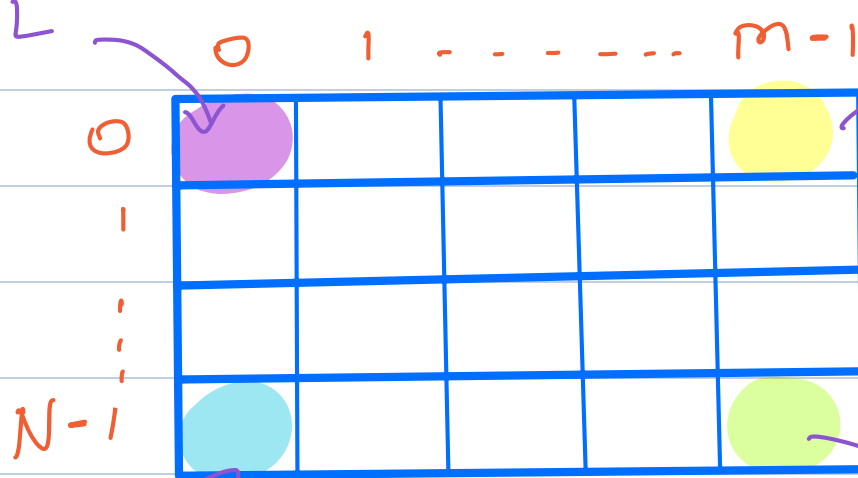
mat[0][0].



Quiz 1

mat[0][0].

TL



Quiz 2

TR

mat[0][m-1].

BR

mat[N-1][m-1].

B.L.

mat[N-1][0]

Q: 1. Given a matrix, print row-wise sum.

ex:- mat[3][4];

		col				<u>Output</u>
		0	1	2	3	
row	0	1	2	3	4	10
	1	5	6	7	8	26
	2	9	10	11	12	42

$$\begin{aligned} \text{Sum} &= 0 + 1 + 2 + 3 + 4 && 10 \\ &= 0 + 5 + 6 \end{aligned}$$

Approach :

```
void rowSum (int mat[][ ], N, m) {  
    int sum;  
    for (int row = 0; row < N; row++) {  
        sum = 0;  
        for (int col = 0; col < m; col++) {  
            sum += mat[row][col];  
        }  
        print(sum);  
    }  
}
```

3
3
3

T.C $\rightarrow O(N \times m)$

S.C $\rightarrow O(1)$

Q:- 3

Q:- Given a matrix, print col-wise sum.

ex:-

	0	1	2	3	
0	1	2	3	4	15
1	5	6	7	8	18
2	9	10	11	12	21
					24

```
void colSum (int mat[][ ], N, m) {
```

```
    int sum;

    for (int col = 0; col < m; col++) {
        sum = 0;
        for (int row = 0; row < N; row++) {
            sum += mat[row][col];
        }
        print (sum);
    }
}
```

T.C $\rightarrow O(N \times m)$

S.C $\rightarrow O(1)$.

Q:- Given a square matrix, print its diagonals
 $N = m$
 $i + j = N - 1$
 $j = \underline{N - 1 - i}$

$i = j$

	0	1	2	
0	1	5	6	0, 0
1	4	3	1	1, 1
2	6	5	2	2, 2

Principal Diagonal

	0	1	2	
0	1	5	6	0, 2
1	4	3	1	1, 1
2	6	5	2	2, 0

Principal Anti Diagonal.

```
void printDiagonals (int mat[][N], N) {
```

Print Principal Diagonal.

```
for (i = 0; i < N; i++) {  
    print (mat[i][i]);
```

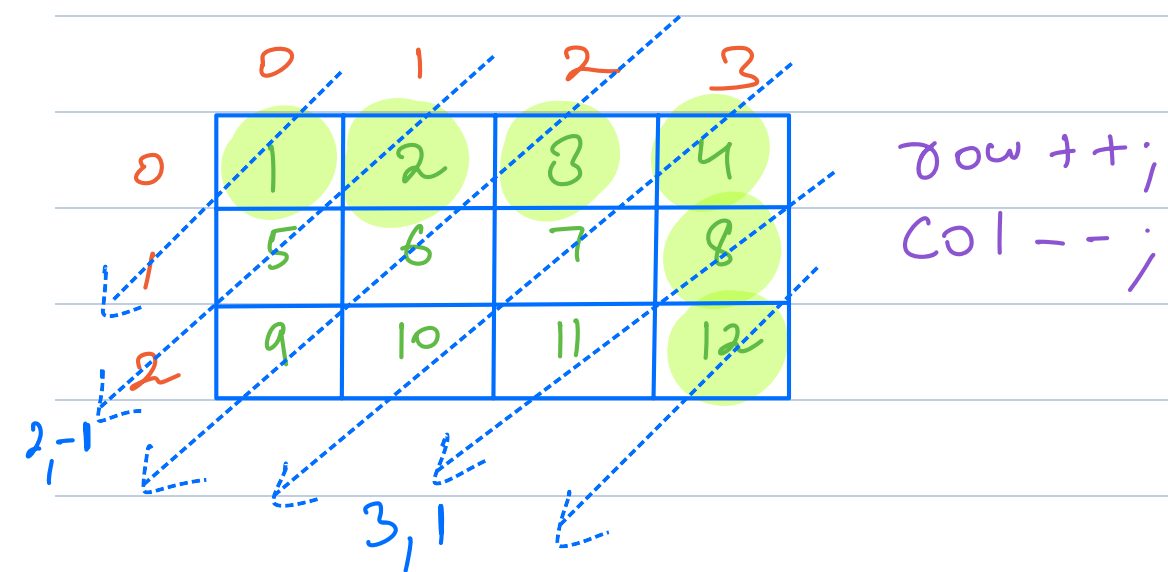
Print Principal Anti Diagonal.

```
for (i = 0; i < N; i++) {  
    print (mat[i][N - 1 - i]);
```

T.C $\rightarrow O(N)$; S.C $\rightarrow O(1)$

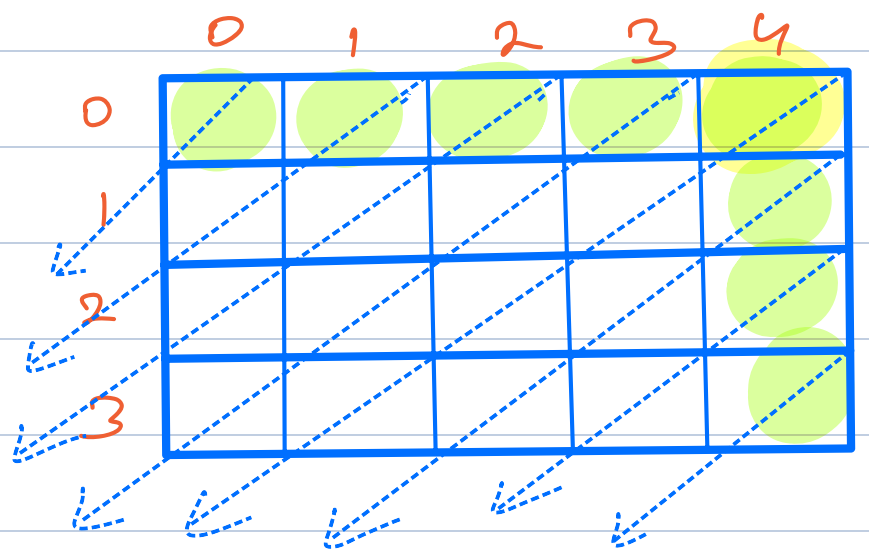
Q:- Print diagonals in a matrix
(right to left)

Output:-



1
2 5
3 6 9
4 7 10
8 11
12

Quiz 5 $4 + 5 - 1 = 8$



Quiz 6 $m + n - 1$

$5 + 4 - 1 = 8$

Observations:-

1. All diagonals follow the same movement pattern of $row++$ & $col--$;
2. All diagonals either start from the first row or from the last col. making total count as $N+m-1$.

1. Fix the S.P. of first row. Move a diagonal until there is a valid cell.

Repeat the process by fixing S.P. in the last col.

Code

$i, j \rightarrow$ S.P. of a diagonal
 $\text{row}, \text{col} \rightarrow$ For every m that diagonal.

```
void printAllDiagonals (int mat[N][M], N, M) {
```

Fix the S.P. of first row.

```
    i = 0;
    for (int j = 0; j < M; j++) {
        row = i; col = j;
        while (row < N && col >= 0) {
            print (mat[row][col];
            row++; col--;
        }
        println();
    }
```

Fix the S.P. of last column.

```
    j = M-1;
    for (i = 1; i < N; i++) {
        row = i; col = j;
        while (row < N && col >= 0) {
            print (mat[row][col];
            row++; col--;
        }
        println();
    }
```

8:25

TC $\rightarrow O(N \times M)$
SC $\Rightarrow O(1)$

Q:- Given a 2D matrix A.
 If a cell $A[i][j] = 0$, make
 entire row i as 0 & entire
 col j as 0.

Ex:-

	0	1	2	3
0	1	2	3	4
1	5	6	7	0
2	9	2	0	4

	0	1	2	3
0	1	2	0	0
1	0	0	0	0
2	0	0	0	0

i/p:

o/p:-

	0	1	2	3
0	1	2	3	4
1	5	6	7	0
2	9	2	0	4

Ex:- 2:-

	0	1	2	3
0	1	2	3	4
1	5	0	7	3
2	9	2	2	4

Approach:-

In first iteration, simply mark
 the presence of a zero.

In second iteration, you refer the
 markers & update the cells.

	0	1	2	3
0	1	2	3	4
1	5	6	7	0
2	9	2	0	4

row: 0, 1, 2, 3
 col: 0, 1, 2, 3

```
int row[N];  // row[i] = 0;
int col[m];  // col[j] = 0;
```

1st iteration

```
for (i = 0; i < N; i++) {
  for (j = 0; j < m; j++) {
    if (mat[i][j] == 0) {
      row[i] = 1;
      col[j] = 1;
    }
  }
}
```

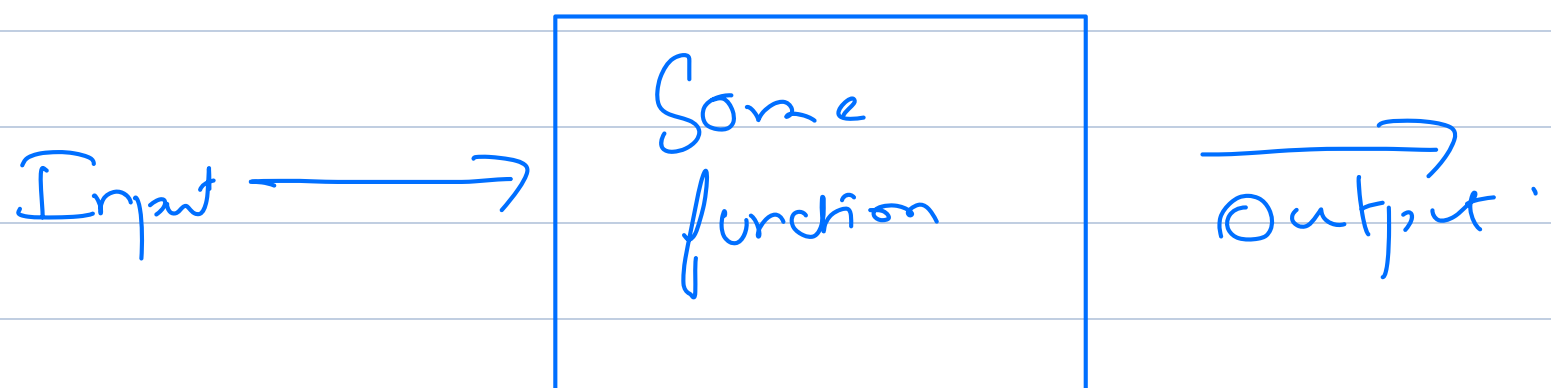
2nd iteration

```
for ( i = 0; i < N; i ++ ) {  
    for ( j = 0; j < m; j ++ ) {  
        if ( row[i] == 1 || col[j] == 1 ) {  
            mat[i][j] = 0;  
        }  
    }  
}
```

T.C $\rightarrow O(N \times m)$

S.C $\rightarrow O(N + m);$

8:45



↓
Any other memory
my function user is str.